Arquitectura del sistema: Ventas e Inventario (CD → Locales)

TP Integrador 2026, Tema 3. Desarrollo de Aplicaciones con Objetos.

> **Alcance de este documento:** describe la estructura, las responsabilidades y las reglas del sistema. **No contiene implementación de métodos**: solo atributos, responsabilidades y contratos. Los **patrones de diseño no están fijados**: cada punto donde podría aplicarse uno está marcado como **[PUNTO DE EXTENSIÓN]** con opciones candidatas, para decidir cuando los docentes los asignen.

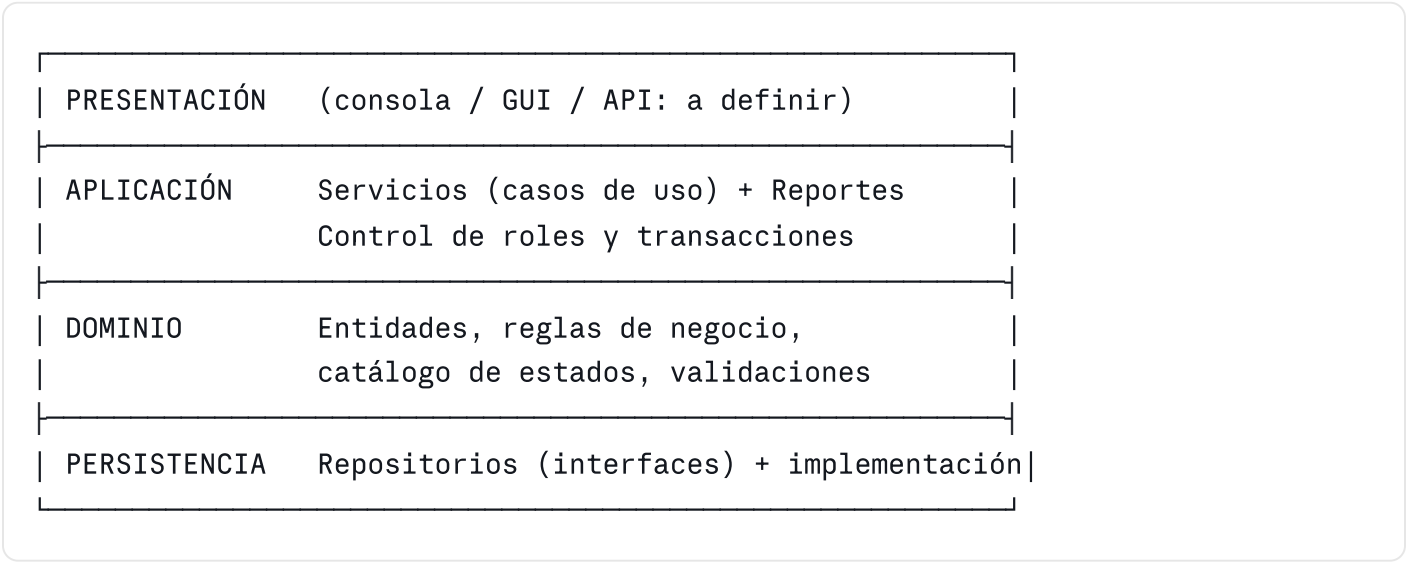
---

1. Principios de arquitectura

1. **Dominio independiente de todo lo demás.** Las clases del dominio no conocen la interfaz, la base de datos ni los reportes.
2. **Capas con dependencias en un solo sentido:** presentación → aplicación → dominio ← persistencia.
3. **Historial inmutable de estados.** Un cambio de estado cierra el registro vigente (`fechaHoraFin`) y abre uno nuevo. Nunca se edita ni se borra un registro previo.
4. **Los puntos de variación se aíslan detrás de interfaces.** Así el patrón elegido por los docentes se aplica sin reescribir el resto.
5. **Operaciones críticas atómicas.** Toda operación que toca varias entidades (verificar un bulto, recibir una compra, registrar una devolución) se completa entera o no se aplica.
6. **Stock con dos magnitudes:** `existencia` (física contable) y `reservado` (comprometido por bultos en preparación o tránsito). `disponible = existencia - reservado`.

---

2. Vista de capas



Capa	Responsabilidad	No debe hacer
Presentación	Capturar datos, mostrar resultados, invocar servicios	Reglas de negocio
Aplicación	Orquestar casos de uso, abrir/cerrar transacciones, verificar permisos por rol	Guardar estado propio del negocio
Dominio	Invariantes, transiciones válidas, cálculos	Acceder a base de datos o interfaz
Persistencia	Guardar y recuperar entidades	Decidir reglas de negocio

2.1 Estructura de paquetes sugerida

```
src/
├── dominio/
|   ├── ubicaciones/      Ubicacion, CentroDistribucion, LocalMinorista
|   ├── catalogo/         Producto, CategoriaProducto, Proveedor
|   ├── inventario/       ItemInventario, Merma
|   ├── compras/          Compra, DetalleCompra
|   ├── logistica/        OrdenReposicion, DetalleOrdenReposicion, Remito, Bulto
|   ├── estados/          Estado, EstadoActual, catálogo y transiciones
|   ├── ventas/           Venta, DetalleVenta, Comprobante, TipoComprobante,
|   |                     MedioDePago, Devolucion, DetalleDevolucion
|   ├── caja/             Caja, MovimientoCaja, TipoMovimiento
|   └── seguridad/        Usuario, Rol
├── aplicacion/
|   ├── servicios/        un servicio por caso de uso (sección 6)
|   └── reportes/         los 4 reportes obligatorios
├── persistencia/
|   ├── repositorios/     interfaces
|   └── implementacion/   en memoria / archivo / base de datos (a definir)
├── presentacion/
└── configuracion/        parámetros (plazo de devolución, etc.)
```

---

3. Modelo de dominio

Convenciones: `1`, `0..1`, `1..*`, `0..*` indican multiplicidad. Todas las fechas con hora son `DateTime`. Los nombres están corregidos respecto de los borradores (`str`, `producto`).

3.1 Ubicaciones y seguridad

Ubicacion (abstracta)

- `idUbicacion : int`
- `nombre : str`
- `direccion : str`
- `telefono : str`

CentroDistribucion (hereda de Ubicacion): único en el sistema. No tiene caja.

LocalMinorista (hereda de Ubicacion)

- `caja : Caja (1)`

Rol

- `idRol : int`
- `nombreRol : str` → valores: `ADMIN_CD`, `ADMIN_MINORISTA`, `VENDEDOR`

Usuario

- `idUsuario : int`
- `nombre : str`
- `rol : Rol (1)`
- `ubicacion : Ubicacion (1)`

3.2 Catálogo

Proveedor

- `idProveedor : int`
- `nombre : str`
- `telefono : str`
- `direccion : str`
- `cuit : str` (identificador único, evita duplicados)

CategoriaProducto

- `idCategoriaProducto : int`
- `descripcion : str`

Producto

- `idProducto : int`

- `sku : str` (único, evita duplicados)
- `nombre : str`
- `precioUnitario : float`
- `proveedor : Proveedor (1)`
- `categoria : CategoriaProducto (1)`

3.3 Inventario

ItemInventario (clave única: `producto` + `ubicacion`)

- `producto : Producto (1)`
- `ubicacion : Ubicacion (1)`
- `existencia : int`
- `reservado : int` (solo se usa en el CD; en locales vale 0)
- `minimo : int`

Merma *(recomendada, opcional)*: registro unificado para que los reportes no mezclen orígenes.

- `idMerma : int`
- `producto : Producto (1)`
- `cantidad : int`
- `fecha : DateTime`
- `origen : OrigenMerma (1)` → `LOGISTICA_PRECINTO`, `LOGISTICA_OTRO`, `DEVOLUCION_NO_APTA`
- `bulto : Bulto (0..1)`
- `detalleDevolucion : DetalleDevolucion (0..1)`

3.4 Compras

Compra

- `idCompra : int`
- `fechaEmision : DateTime`
- `proveedor : Proveedor (1)`
- `centroDistribucion : CentroDistribucion (1)`
- `usuario : Usuario (1)`
- `estados : EstadoActual (1..*)`
- `detallesCompra : DetalleCompra (1..*)`

DetalleCompra

- `producto : Producto (1)`
- `cantidad : int`
- `precioCosto : float`

Invariante: todos los productos de la compra pertenecen al `proveedor` de la compra.

3.5 Logística

OrdenReposicion

- `idOrden : int`
- `fechaSolicitud : DateTime`
- `origen : CentroDistribucion (1)`
- `destino : LocalMinorista (1)`
- `usuario : Usuario (1)`
- `estados : EstadoActual (1..*)`
- `detallesOrdenReposicion : DetalleOrdenReposicion (1..*)`
- `remitos : Remito (0..*)` *(ver decisión D-2 en la sección 10)*

DetalleOrdenReposicion

- `producto : Producto (1)`
- `cantidad : int`

Remito

- `idRemito : int`
- `fechaDespacho : DateTime`
- `orden : OrdenReposicion (1)`
- `bultos : Bulto (1..*)`
- `estados : EstadoActual (1..*)` *(ver decisión D-1)*

Bulto

- `idBulto : int`
- `precintoSeguridad : str`
- `producto : Producto (1)` (un único producto por bulto)
- `cantidad : int`
- `detalleOrden : DetalleOrdenReposicion (1)` (1 detalle → N bultos)
- `remito : Remito (0..1)` (se asigna al precintar)
- `fechaHoraRecepcion : DateTime (0..1)`

- `motivoRechazo : MotivoRechazo (0..1)` → `PRECINTO_VIOLADO`, `ENVIO_ERRONEO`, `OTRO`
- `observacion : str (0..1)`
- `estados : EstadoActual (1..*)`

3.6 Estados (patrón de estado con historial)

Estado (catálogo)

- `descripcion : str`
- `ambito : str` → `BULTO`, `REMITO`, `ORDEN_REPOSICION`, `COMPRA`

EstadoActual (cada transición registrada)

- `estado : Estado (1)`
- `fechaHoraInicio : DateTime`
- `fechaHoraFin : DateTime (0..1)` (nulo = estado vigente)
- `usuario : Usuario (1)` *(recomendado: quién hizo la transición)*

Regla: `Estado.ambito` debe coincidir con el tipo de objeto que posee el `EstadoActual`.

3.7 Ventas, caja y devoluciones

Venta

- `idVenta : int`
- `fecha : DateTime`
- `caja : Caja (1)`
- `medioDePago : MedioDePago (1)`
- `usuario : Usuario (1)`
- `comprobante : Comprobante (0..1)`
- `detallesVenta : DetalleVenta (1..*)`

DetalleVenta

- `itemInventario : ItemInventario (1)`
- `cantidad : int`
- `precioUnitario : float` (precio congelado al momento de la venta)

MedioDePago: `idMedioDePago : int`, `descripcion : str`

Comprobante

- `idComprobante : int`

- `total : float`
- `fecha : DateTime`
- `tipo : TipoComprobante (1)`

TipoComprobante: `idTipoComprobante : int`, `descripcion : str` → `TICKET_FACTURA`, `NOTA_CREDITO`

Caja

- `idCaja : int`
- `saldo : float`
- `movimientosCaja : MovimientoCaja (0..*)`

MovimientoCaja

- `idMovimiento : int`
- `monto : float`
- `fecha : DateTime`
- `tipo : TipoMovimiento (1)`
- `venta : Venta (0..1)`
- `devolucion : Devolucion (0..1)`

Exactamente uno de `venta` o `devolucion` está informado.

TipoMovimiento: `idTipoMovimiento : int`, `descripcion : str` → `VENTA` (entrada), `DEVOLUCION` (salida)

Devolucion

- `idDevolucion : int`
- `fecha : DateTime`
- `motivo : str` (obligatorio)
- `venta : Venta (1)`
- `notaCredito : Comprobante (1)` (de tipo `NOTA_CREDITO`)
- `usuario : Usuario (1)`
- `detalles : DetalleDevolucion (1..*)`

DetalleDevolucion

- `detalleVenta : DetalleVenta (1)`
- `cantidad : int`
- `aptoParaVenta : bool`

3.8 Diagrama de relaciones (resumen)

```
Ubicacion ◁── CentroDistribucion
          ◁── LocalMinorista ──1── Caja ──0..*── MovimientoCaja
Usuario ──1── Rol
Usuario ──1── Ubicacion

Proveedor ──1..* Producto ──1── CategoriaProducto
Producto ──*── ItemInventario ──*── Ubicacion

Compra ──1── Proveedor
Compra ──1..*── DetalleCompra ──1── Producto
Compra ──1..*── EstadoActual ──1── Estado

OrdenReposicion ──1..*── DetalleOrdenReposicion ──1── Producto
OrdenReposicion ──0..*── Remito ──1..*── Bulto ──1── DetalleOrdenReposicion
Bulto / Remito / OrdenReposicion ──1..*── EstadoActual

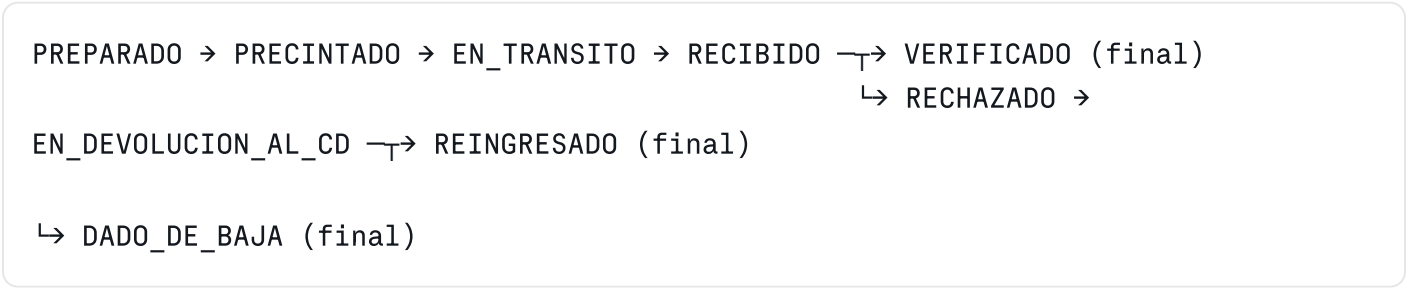
Venta ──1..*── DetalleVenta ──1── ItemInventario
Venta ──0..1── Comprobante ──1── TipoComprobante
Devolucion ──1── Venta
Devolucion ──1..*── DetalleDevolucion ──1── DetalleVenta
Devolucion ──1── Comprobante (NOTA_CREDITO)
MovimientoCaja ──0..1── Venta | Devolucion
```

---

4. Ciclos de vida (catálogo de estados)

4.1 Ámbito `BULTO`

Estado	Significado	Registra
PREPARADO	El CD armó el bulto (producto y cantidad). Reserva stock.	ADMIN_CD
PRECINTADO	Se colocó el precinto y se asignó a un remito	ADMIN_CD
EN_TRANSITO	Se registró el despacho; salió del CD	ADMIN_CD
RECIBIDO	Llegó al local; precinto sin controlar	ADMIN_MINORISTA
VERIFICADO	Precinto íntegro. **Estado final.** Stock disponible en el local	ADMIN_MINORISTA
RECHAZADO	No se aceptó (precinto roto u otro motivo). Sigue visible para el reporte 4	ADMIN_MINORISTA
EN_DEVOLUCION_AL_CD	El bulto rechazado vuelve al CD	ADMIN_MINORISTA / ADMIN_CD
REINGRESADO	El CD lo revisó y libera el stock. **Estado final**	ADMIN_CD
DADO_DE_BAJA	El CD lo revisó y no es recuperable (merma). **Estado final**	ADMIN_CD



No se puede saltear ni retroceder ningún paso. Si el rechazo fue por `PRECINTO_VIOLADO`, el CD debe revisar el contenido antes de reingresar o dar de baja. Si fue por otro motivo no ligado a la integridad (ej. envío erróneo), normalmente va a `REINGRESADO`.

4.2 Ámbito `REMITO` *(ver decisión D-1)*

`EMITIDO → EN_TRANSITO → RECIBIDO → VERIFICADO`

El remito pasa a `VERIFICADO` cuando todos sus bultos están resueltos y al menos uno quedó `VERIFICADO`. Alternativa: derivar el estado de los bultos sin guardarlo.

4.3 Ámbito `ORDEN_REPOSICION`

`PENDIENTE → EN_PREPARACION → DESPACHADA → FINALIZADA`, y `PENDIENTE` o `EN_PREPARACION → CANCELADA`.

`FINALIZADA` cuando todos sus bultos están en un estado final (`VERIFICADO`, `REINGRESADO` o `DADO_DE_BAJA`). No se cancela una orden ya `DESPACHADA`.

4.4 Ámbito `COMPRA`

`EMITIDA → RECIBIDA → VERIFICADA`, y `EMITIDA → CANCELADA`.

`RECIBIDA` indica que la mercadería llegó. `VERIFICADA` indica que se controló contra lo pedido; recién ahí suma al stock del CD.

4.5 Sin estados propios

`Venta` y `Devolucion` se registran en un solo paso y son inmutables. Su historia posterior son registros nuevos (devoluciones, movimientos de caja).

4.6 [PUNTO DE EXTENSIÓN 1] Control de transiciones

La lógica de "qué transición es válida y qué efecto tiene" debe estar **detrás de una interfaz** (`ControlDeTransiciones` o equivalente), no dispersa en `if` dentro de las entidades.

Opción	Cómo encaja
State	Una clase por estado del bulto que sabe a qué estados puede ir y qué efectos dispara
Strategy	Una estrategia por ámbito (bulto, orden, compra) que valida y ejecuta la transición
Tabla de transiciones (data-driven)	Mapa `estado → estados permitidos` cargado desde el catálogo
Command	Cada transición es un comando con su efecto y su validación

---

5. Reglas de stock (definidas por el grupo)

5.1 Efecto de cada transición

Evento	`existencia` CD	`reservado` CD	`existencia` local	Otros
Compra `RECIBIDA`	sin cambio	sin cambio	sin cambio	Solo registra llegada
Compra `VERIFICADA`	+ **cantidad**	sin cambio	sin cambio	Habilita stock del CD
Bulto `PREPARADO`	sin cambio	+ **cantidad**	sin cambio	Valida `disponible ⩾ cantidad`
Bulto `EN_TRANSITO`	sin cambio	sin cambio	sin cambio	Solo registra despacho
Bulto `VERIFICADO`	− **cantidad**	− **cantidad**	+ **cantidad**	Operación atómica
Bulto `RECHAZADO`	sin cambio	sin cambio	sin cambio	Registra motivo
Bulto `REINGRESADO`	sin cambio	− **cantidad**	sin cambio	Libera la reserva; el stock vuelve a estar disponible
Bulto `DADO_DE_BAJA`	− **cantidad**	− **cantidad**	sin cambio	Genera `Merma`
Venta	sin cambio	sin cambio	− **cantidad**	Solo si `cantidad ⩽ existencia` del local
Devolución apta	sin cambio	sin cambio	+ **cantidad**	En el local donde se vendió
Devolución no apta	sin cambio	sin cambio	sin cambio	Genera `Merma`

5.2 Consecuencias

- **El stock nunca sale del CD hasta que el local verifica.** Mientras tanto queda reservado, de modo que no puede asignarse a otro bulto.
- **Una venta solo ve stock verificado:** el local solo suma existencia en `VERIFICADO`.

- **Un rechazo no pierde stock:** como nunca se restó del CD, solo se libera la reserva (`REINGRESADO`).
- **Mermas separadas por origen**, para que el reporte 4 (rotura de precinto) no se mezcle con las mermas por devolución.

---

6. Capa de aplicación: servicios (casos de uso)

Cada servicio expone casos de uso, valida permisos por rol y define el límite de la transacción. **No se detalla la implementación.**

Servicio	Proceso del enunciado	Rol	Casos de uso
`ServicioUbicaciones`	1	ADMIN_CD	Alta/baja/modificación de CD, locales y cajas
`ServicioCatalogo`	2	ADMIN_CD	Productos, categorías, proveedores, precios, mínimos
`ServicioCompras`	(compras)	ADMIN_CD	Emitir, recibir, verificar, cancelar
`ServicioReposicion`	3	ADMIN_MINORISTA / ADMIN_CD	Solicitar, consolidar necesidades, cancelar, finalizar
`ServicioDespacho`	4 y 5	ADMIN_CD	Armar bultos, precintar, asignar remito, despachar
`ServicioRecepcion`	6	ADMIN_MINORISTA	Registrar recepción, verificar precinto, aceptar o rechazar
`ServicioDevolucionLogistica`	extensión	ADMIN_CD	Gestionar bulto rechazado: reingresar o dar de baja
`ServicioVentas`	7	VENDEDOR	Registrar venta, cobrar, emitir comprobante
`ServicioDevoluciones`	7	VENDEDOR	Validar y registrar devolución, nota de crédito
`ServicioCaja`	7	VENDEDOR	Consultar saldo y movimientos
`ServicioInventario`	transversal	todos	Consultar existencias, reservar, liberar, alertar mínimos

7. Reportes (los 4 obligatorios)

#	Reporte	Filtros	Fuente principal
1	Ventas e ingresos	período, local, medio de pago	`Venta`, `DetalleVenta`, `MedioDePago`, devoluciones
2	Productos más vendidos y margen estimado	período, local, categoría	`DetalleVenta` y costo de `DetalleCompra`
3	Existencias actuales y productos en mínimo	ubicación, categoría	`ItemInventario` (`existencia`, `reservado`, `minimo`)
4	Bultos rechazados y mermas por rotura de precinto	período, local, motivo	`Bulto`, `EstadoActual`, `Merma`

Decisiones de cálculo a fijar (ver sección 10): el costo para el margen (último `precioCosto` o promedio) y si las devoluciones restan de los ingresos del reporte 1.

7.1 [PUNTO DE EXTENSIÓN 2] Generación de reportes

Opción	Cómo encaja
Strategy	Una estrategia de cálculo por reporte, intercambiable
Template Method	Esqueleto común (filtrar → agrupar → calcular → formatear) con pasos específicos por reporte
Builder	Para armar el resultado/estructura del reporte paso a paso
Visitor	Si se recorren colecciones de entidades con operaciones distintas

8. Puntos de extensión para patrones de diseño

La consigna exige **al menos 2 patrones**. Esta tabla muestra dónde cada patrón aporta valor real, para elegir según lo que indiquen los docentes.

#	Punto de variación	Qué problema resuelve	Patrones candidatos
1	Control de transiciones de estado (sección 4.6)	Evitar `if` repetidos y transiciones inválidas	State, Strategy, Command, tabla de transiciones
2	Generación de reportes (sección 7.1)	Reportes intercambiables con estructura común	Strategy, Template Method, Builder
3	Aviso de stock mínimo	Reaccionar cuando `existencia ≤ minimo` sin acoplar `ItemInventario` al aviso	Observer
4	Creación de comprobantes (ticket/factura vs. nota de crédito)	Elegir el tipo y construirlo con sus reglas	Factory Method, Abstract Factory, Builder
5	Acceso a datos	Aislar el dominio de la persistencia	Repository, DAO
6	Entrada única a los casos de uso	Ocultar la coordinación de varios servicios	Facade
7	Parámetros globales (plazo de devolución, etc.)	Un único punto de configuración	Singleton (con precaución), inyección de dependencias
8	Reglas de validación de devolución	Validaciones combinables y ordenadas	Chain of Responsibility, Specification, Strategy
9	Cálculo del costo para el margen	Alternar entre último costo y promedio	Strategy
10	Auditoría de operaciones críticas	Registrar quién hizo qué sin ensuciar el dominio	Decorator, Observer

Combinaciones razonables según lo que pidan los docentes:

- Si piden **State**: se aplica al ciclo del bulto (punto 1). Es el que tiene más valor por la cantidad de estados.
- Si piden **Observer**: stock mínimo (punto 3).
- Si piden **Strategy**: reportes o costo del margen (puntos 2 o 9).
- Si piden **Factory**: comprobantes (punto 4).
- Si piden **Repository**/**Facade**: puntos 5 y 6, que no alteran el dominio.

Ningún punto del dominio depende de un patrón concreto: cada uno se alcanza por una interfaz que puede implementarse de más de una forma.

---

9. Reglas de negocio y dónde se validan

9.1 Validaciones del enunciado

Regla	Dónde se hace cumplir
El stock no impacta en el local hasta `VERIFICADO`	Dominio: transición del bulto (5.1)
Precinto roto → `RECHAZADO`, no habilita venta	Dominio: transición del bulto
Trazabilidad inmutable de estados	Dominio: `EstadoActual` solo se cierra y se agrega
No vender más que el stock disponible y verificado	Aplicación (`ServicioVentas`) + dominio (`ItemInventario`)
Sin productos ni proveedores duplicados	Persistencia (clave única: `sku`, `cuit`) + aplicación
Devoluciones solo de ventas registradas	Dominio (`Devolucion.venta` obligatorio)

9.2 Reglas propias del grupo

- Cada producto tiene un único proveedor.
- Hay un solo CD y una sola caja por local; el CD no tiene caja.
- Cada bulto contiene un único producto.
- Todos los productos de una compra son del mismo proveedor.
- `(producto, ubicacion)` es único en `ItemInventario`.
- `venta.caja` debe pertenecer a `usuario.ubicacion` (un vendedor no vende en la caja de otro local).
- El `Estado.ambito` debe coincidir con el tipo de objeto.
- `MovimientoCaja` referencia exactamente una venta o una devolución.

9.3 Reglas de devolución

1. Debe corresponder a una venta registrada y hacerse en el mismo local.
2. Se puede devolver sin comprobante, identificando la venta por `idVenta`.
3. Plazo máximo configurable desde la venta (`configuracion`, valor a definir).
4. El motivo es obligatorio.

5. Cada renglón referencia un `DetalleVenta` de esa venta.
6. La cantidad devuelta acumulada por renglón no supera la vendida.
7. El monto se calcula con el `precioUnitario` del `DetalleVenta`.
8. Genera siempre una nota de crédito por el total devuelto.
9. `aptoParaVenta = true` suma al `ItemInventario` del local; `false` genera `Merma`.
10. Genera un movimiento de caja de salida por el monto.
11. El reintegro usa el medio de pago original (se toma de `venta.medioDePago`).
12. *(A decidir)* Rechazar si el saldo de caja es insuficiente.
13. La venta original no se modifica; la devolución es inmutable.

9.4 Atomicidad

Deben ejecutarse como una sola unidad: verificar bulto (3 magnitudes de stock + estado + remito), verificar compra, registrar venta (stock + caja + comprobante), registrar devolución (stock o merma + nota + caja).

---

10. Decisiones abiertas

ID	Decisión	Opciones	Recomendación
D-1	¿`Remito` lleva estados propios?	(a) Ámbito `REMITO` propio. (b) Derivarlo de sus bultos	(a), porque el enunciado dice que "remito y contenedor pasan a Verificado"
D-2	¿Una orden tiene uno o varios remitos?	(a) Uno. (b) Varios (despachos parciales)	(a) para simplificar, salvo que el CD despache por partes
D-3	¿Merma como entidad propia?	(a) Clase `Merma`. (b) Derivar de estados y `aptoParaVenta`	(a), simplifica el reporte 4 y separa orígenes
D-4	Faltante por rechazo	(a) Genera nueva orden. (b) Queda pendiente en la misma	A decidir; afecta el estado `FINALIZADA`
D-5	Saldo de caja insuficiente en devolución	(a) Bloquear. (b) Permitir saldo negativo	(a); definirlo porque afecta los tests
D-6	Costo para margen estimado	(a) Último `precioCosto`. (b) Promedio ponderado	(a) por simplicidad, con Strategy si piden flexibilidad
D-7	Plazo de devolución	Número de días	30, como parámetro
D-8	¿La devolución resta del ingreso en el reporte 1?	(a) Sí (neto). (b) Se muestra aparte	(b), más transparente
D-9	Tecnología de persistencia	Memoria, archivos, SQLite, otra	A definir; la arquitectura lo permite por el Repository
D-10	Tecnología de presentación	Consola, GUI, API	A definir

11. Trazabilidad contra el enunciado

Requisito	Cubierto en
Proceso 1: ubicaciones	3.1, `ServicioUbicaciones`
Proceso 2: productos, categorías, proveedores, mínimos	3.2, 3.3, `ServicioCatalogo`
Proceso 3: orden de reposición	3.5, 4.3, `ServicioReposicion`
Proceso 4: bultos, remito, precinto	3.5, 4.1, `ServicioDespacho`
Proceso 5: despacho	4.1, 5.1, `ServicioDespacho`
Proceso 6: recepción y verificación	4.1, 5.1, `ServicioRecepcion`
Proceso 7: ventas, cobros, comprobante, devoluciones	3.7, 9.3, `ServicioVentas`, `ServicioDevoluciones`
Validaciones de negocio (5)	9.1
Reportes (mínimo 4 no triviales)	7
Al menos 2 patrones de diseño	8 (a definir con los docentes)
Ciclo del bulto evaluado críticamente	4.1, 5
Procesos opcionales	Fuera de alcance
Reportes de tránsito y valorización	Fuera de alcance (no obligatorios)

Extensiones respecto del enunciado (a justificar ante los docentes)

1. **Estados `EN_DEVOLUCION_AL_CD`, `REINGRESADO` y `DADO_DE_BAJA`** en el ámbito BULTO: evitan perder stock válido tras un rechazo, manteniendo `RECHAZADO` visible para el reporte 4.
2. **Estado `VERIFICADA` en compras:** el enunciado solo exige verificación en el flujo CD → local; el grupo la extiende a compras.
3. **Campo `reservado` en `ItemInventario`:** consecuencia de no restar stock del CD hasta la verificación; evita doble asignación.
4. **Entidad `Merma`:** separa mermas logísticas de mermas por devolución.